

mapc2022_pcs5703

Esqueleto JaCaMo para o 2º Exercício Prático de PCS 5703 — Sistemas Multi-Agentes (1º Quadrimestre de 2026, Escola Politécnica da USP).

Implementa um time de agentes para o **Multi-Agent Programming Contest 2022** (cenário *Agents Assemble II*), conforme exigido pelo enunciado, usando:

- **Jason** (plataforma de agentes BDI) — referência [4]
- **MOISE+** (modelo organizacional) — referência [3]
- **JaCaMo** (arcabouço integrador) — referência [2]
- **EISMASSim** (ponte EIS para o servidor MASSim 2022) — referência [6]

Estrutura do projeto

```
mapc2022_pcs5703/
├─ mapc2022.jcm          # arquivo principal: declara MAS, agentes, env e org
├─ build.gradle          # build Gradle (chama jacamo.infra.JaCaMoLauncher)
├─ settings.gradle
├─ conf/
│   └─ eismassimconfig.json # config da ponte EIS<->MASSim
├─ lib/
│   └─ eismassim.jar       # eismassim-4.5-jar-with-dependencies copiado
└─ src/
    ├─ agt/                # agentes Jason (.asl)
    │   └─ agente_base.asl # planos comuns a todos
    │   └─ coordenador.asl # papel social MOISE+ "coordinator"
    │   └─ explorador.asl  # papel social MOISE+ "explorer"
    ├─ env/                # artefatos CArtAg0 (Java)
    │   └─ mapc/
    │       └─ QuadroEquipe.java
    ├─ org/                # especificação MOISE+
    │   └─ org.xml
    └─ java/               # código Java auxiliar
        └─ jason/eis/
            └─ EISAdapter.java # ponte Jason<->EISMASSim
```

Pré-requisitos

- JDK ≥ 17 (verifique: `java -version`)
- O servidor MASSim 2022 já compilado (em `../massim_2022/`, basta rodar `mvn package` lá uma vez)

Como rodar

1. Suba o servidor MASSim em outro terminal

```
cd ../massim_2022
java -jar server/target/server-2022-1.1-jar-with-dependencies.jar \
    -conf server/conf/SampleConfig.json --monitor 8000
```

Servidor escutará em `localhost:12300`. Monitor web em <http://localhost:8000/>.

2. Suba o time JaCaMo

```
cd mapc2022_pcs5703
./gradlew run
```

A primeira execução baixa o JaCaMo 1.3.0 do Maven (≈ 2 min). Depois é instantânea.

3. Inicie a partida no servidor

No terminal do servidor, pressione **ENTER** quando os 3 agentes do `mapc2022_pcs5703` (e idealmente os 3+ do time adversário) estiverem autenticados.

Mapeamento Jason ↔ servidor

Agente Jason (.jcm)	Entidade EIS (eismassimconfig.json)	Username MASSim
---------------------	-------------------------------------	-----------------

coordenador1	coordenador1	agentA1
--------------	--------------	---------

explorador1	explorador1	agentA2
-------------	-------------	---------

explorador2	explorador2	agentA3
-------------	-------------	---------

A senha de todos é `1` (vem do `SampleConfig.json` do servidor). Para usar o time **B** em vez do **A**, troque `agentA*` por `agentB*` no `eismassimconfig.json`.

Como o servidor do contest (MASSim 2022) é configurado

Toda a configuração é feita por **arquivos JSON**. Um config de torneio (ex.: `server/conf/SampleConfig.json`) referencia uma **lista de simulações**, cada uma num arquivo em `server/conf/sim/`. O torneio roda esses sims **em sequência**, podendo cada um ter regras diferentes. Cada simulação define:

- **grid** (largura $\times$ altura) e os obstáculos/cavernas;

- **agentes por time** (`entities`) e quantos times competem (`teamsPerMatch`);
- **dispensers, goal zones e role zones** (quantidade);
- **tasks**: tamanho (nº de blocos), quantas simultâneas, reward, deadline;
- **papéis** (`default/worker/digger/...`), cada um com suas ações, visão e velocidade;
- **normas, eventos** (`clear events`) e o número de **passos**.

Os parâmetros **variam entre simulações** — não são fixos. Exemplos dos sims de amostra que acompanham o servidor:

Sim	Grid	Agentes/time	Dispensers	Goal/Role zones	Tasks	Passos
sim1	70×70	20	5–10	4 / 5	tamanho 1–4, 2 simultâneas	750
sim2	100×100	40	5–10	7 / 5	tamanho 2–4, 3 simultâneas	750

Pontos importantes para a estratégia:

- **Dois times competem** (`teamsPerMatch: 2`), cada um com **20–40 agentes**. Este esqueleto controla apenas **3 agentes** (declarados no `.jcm/eismassimconfig.json`) — para escalar, declare mais entidades.
- **O grid é TOROIDAL**: ao sair por uma borda, o agente reaparece na borda oposta. Isso afeta diretamente o rastreio de posição absoluta por soma de deslocamentos (a coordenada cresce sem limite a cada volta) — uma solução robusta precisa de **coordenadas conscientes do tamanho do grid** (módulo).
- As percepções são **relativas** (`absolutePosition: false`); o agente não recebe sua posição absoluta nem o tamanho do grid — precisa inferir/mapear.

Alcance de visão por papel (e como foi determinado)

A visão é um **raio em distância de Manhattan**: um agente percebe toda célula a no máximo `r` passos de si (`scenario.md §"Vision range"`). O valor de `r` é **propriedade do papel**, definido em `server/conf/sim/roles/standard.json`:

Papel	vision na config
default	5
explorer	7
worker, constructor, digger	(ausente)

Surge a dúvida: se `worker` **não** declara `vision`, ele fica **cego** (visão 0) ao ser adotado? Isso é decisivo para a estratégia, pois um worker cego não conseguiria navegar até dispensers/goal zones por percepção.

Método de verificação: inspeção do código-fonte do servidor

(`massim_2022/protocol/src/main/java/massim/protocol/data/Role.java`). O parser de papéis lê a visão com

```
jsonRole.optInt("vision", baseRole.vision)
```

ou seja, quando a chave `vision` está ausente, o papel **herda a visão do papel-base** (o default, `vision = 5`). Confirmado também por `Entity.getVision()`, que retorna `this.role.vision()`.

Conclusão: `worker/constructor/digger` têm **visão 5** (herdada), não são cegos. Por isso a estratégia de **navegação por percepção relativa** (mover-se rumo a alvos *visíveis*) funciona inclusive na fase de worker — escolha de projeto adotada justamente por ser robusta ao **grid toroidal** (ver decisão de design abaixo).

Decisões de design e validação (formato de relatório)

Esta seção registra, em estilo de relatório, decisões técnicas tomadas durante o desenvolvimento e como foram validadas em testes ao vivo contra o servidor MASSim 2022. É material de apoio ao Item 5 (estratégia) e Item 6 (características técnicas) do enunciado.

D1 — Navegação robusta ao grid toroidal: percepção relativa em vez de coordenadas absolutas

Contexto. O grid do cenário é **toroidal** (ao sair por uma borda, o agente reaparece na oposta) — verificado empiricamente: num grid 20×20 a posição rastreada chegou a `x = 300` apenas somando *moves* bem-sucedidos. O esqueleto mantém um mapa em **coordenadas absolutas** obtidas pela soma dos deslocamentos a partir do *spawn* (Passo 1).

Problema observado. A navegação gulosa em direção a uma coordenada absoluta gravada **diverge** num toro: a direção "alvo – posição" no referencial ilimitado pode corresponder ao caminho *longo* (dando a volta). O agente então se afasta do alvo; ao se mover, **re-percebe o mesmo alvo numa coordenada ainda mais distante** (o erro acumula), e passa a persegui-lo indefinidamente — comportamento visível como agentes "subindo direto, sumindo numa borda e reaparecendo na oposta". Em mapas grandes com obstáculos o efeito é lento (o agente perambula localmente); em grids pequenos e abertos, explode.

Decisão. Para o trecho final de aproximação, **navegar pela percepção relativa a alvos visíveis** (role zone, dispenser, goal zone): se o alvo está dentro do raio de visão, mover-se rumo ao seu *offset relativo* (`rx, ry`) — que é **sempre o caminho curto correto num toro**, independente de qualquer wrap. O mapa absoluto fica como **guia grosseiro** ("chegar perto") e *fallback* quando nada está visível. Implementado nos planos `acao_worker` do `explorador.asl` (regras `*_visivel` + plano `!mover_rel`).

Limitação conhecida. Uma solução plenamente correta exigiria **coordenadas conscientes do tamanho do grid** (aritmética modular), mas o servidor não fornece o tamanho do grid em modo `absolutePosition: false`; inferi-lo demandaria detecção de *loop closure*. A navegação por visão contorna o problema no que importa (aproximação

final).

D2 — Cenário de teste controlado (DemoConfig.json) para validar o pipeline de tarefa

Contexto. Os mapas do SampleConfig são grandes (70×70, 100×100) e aleatórios, com tarefas de 1–4 blocos. Completar uma tarefa de fato exige a **conjunção** de muitas condições no mesmo episódio (tarefa de 1 bloco *cardinal* + dispenser do tipo descoberto + role zone e goal zone alcançáveis + passos suficientes). Em testes ao vivo, essa conjunção quase nunca se alinhava por acaso, impedindo a **observação** do caminho `request → attach → submit` mesmo com o código implementado.

Decisão. Criar um cenário de demonstração controlado, **aditivo** (não altera os configs originais do servidor), em `massim_2022/server/conf/`:

- `DemoConfig.json` — config de torneio que roda o mesmo sim de demo algumas vezes em sequência (evita reiniciar o servidor a cada teste);
- `sim/sim_demo.json` — grid **30×30** aberto, **apenas tarefas de 1 bloco** (size: [1,1]), ~20 dispensers (todos os 3 tipos), 5 goal zones e 5 role zones, **3 agentes por time**.

Rodar com: `java -jar server/target/server-2022-1.1-jar-with-dependencies.jar -conf server/conf/DemoConfig.json --monitor 8000`.

Resultados dos testes — pipeline completo validado ao vivo. Neste cenário o ciclo de tarefa **fechou ponta a ponta** pela primeira vez. Linha do tempo de um episódio (explorador2):

Passo	Evento	Resultado
—	coordenador seleciona e anuncia <code>task3</code> (bloco <code>b1</code> em <code>(0,1)</code>); explorador se promove a worker	—
20	alcança um role zone e <code>adopt(worker)</code>	success
25	posiciona-se por visão no dispenser e <code>request</code>	success
25	bloco aparece em <code>(0,1)</code> e <code>attach</code>	anexado
35	navega por visão até a goal zone e <code>submit(task3)</code>	success 

Placar do episódio: `adopt 2/2`, `request 2/2`, `attach 2/2`, `submit 1` (o segundo worker também coletou bloco). A navegação é **por visão + exploração** (sem coordenada absoluta no worker, que divergia no toro — ver D1/D3), com uma **camada de mapa** subsidiária que só ruma a um alvo *conhecido* quando ele não está visível e cede para a visão assim que ele entra no raio 5.

Causa-raiz do bloqueio histórico (descasamento átomo × string). O `request → attach → submit` nunca havia

sido observado porque a camada de **visão do worker** comparava o tipo do bloco por **unificação crua**: o tipo da tarefa chega do coordenador via CArtAgO como **string** "b1", mas o tipo no percept `thing(_, _, dispenser, b1)` é um **átomo** `b1` — `"b1" \= b1`, então o worker *nunca reconhecia* o dispenser do tipo certo (embora o **registro/descoberta** funcionasse para os três tipos `b0/b1/b2`, que sempre foram mapeados). Correção: o tipo da tarefa é **convertido a átomo na origem** (`.term2string` com 1º argumento livre, guardado em `tipo_alvo`) e usado por unificação pura em toda a coleta — é o mesmo bug átomo/string já corrigido no coordenador, que faltava replicar aqui. Diagnóstico viabilizado pela instrumentação [DIAGW] (estado da máquina de coleta por passo) e por uma correção de *deadlock* na espera do bloco (timeout `max_espera_bloco`). Após o `submit` bem-sucedido, o worker **libera o estado** (`reset_pos_submit`) e volta a explorar / assumir nova tarefa.

Generalidade aos três tipos de bloco (b0/b1/b2) validada. Como o `randomSeed` do `sim_demo.json` é fixo (a sequência de tarefas é determinística), para exercitar cada tipo de forma controlada adicionou-se ao coordenador uma preferência **opcional** de tipo — `tipo_preferido(T)` (desativada por padrão; em operação normal qualquer tipo serve). Com ela ligada, validou-se ao vivo o ciclo completo para os três tipos: **b0** (`task1`, `submit success` ≈ passo 35), **b1** (`task3`, ≈ passo 35) e **b2** (`task0`, `submit success` nos passos 56 e 59 — **os dois workers** completaram). Isso confirma que a coleta é genérica ao tipo (nada *hardcoded* para `b1`): tanto a descoberta/registro de dispensers quanto o `tipo_alvo` operam sobre o tipo exigido pela tarefa corrente.

D3 — Exploração por cobertura (varredura serpentina) em vez de *random walk*

Contexto. A descoberta de recursos (role zones, dispensers, goal zones) dependia de *random walk*, deixando o sucesso ao acaso — agravado pelo *drift* toroidal quando havia viés direcional. Times fortes do MAPC usam **exploração sistemática** (cobertura), não sorte.

Decisão. Substituir o *random walk* por uma ****varredura serpentina (*lawnmower*) em explorador.asl: cada explorador segue uma direção primária por `cob_leg` passos, dá um passo perpendicular, reverte e repete, varrendo faixas. É robusto ao toro**** (conta passos *relativos*, sem coordenadas absolutas) e os dois exploradores partem de orientações diferentes para cobrir mais. **Efeito medido:** a área coberta cresceu (faixa larga em vez de coluna fina) e a adoção do papel `worker` passou a ocorrer cedo e de forma consistente.

Limitação conhecida. O comprimento da perna (`cob_leg`) idealmente acompanha a largura do grid; como o tamanho do grid não é perceptível, ele é um parâmetro ajustável (heurístico). Para o cenário de demo 30×30, `cob_leg = 20` deu boa cobertura.

D4 — Estratégia para tarefas multi-bloco (`connect`): projeto

Estado: projetado, não implementado. Esta seção documenta o desenho da estratégia para tarefas de 2+ blocos. O pipeline implementado e validado (D2) cobre tarefas de **1 bloco**; o multi-bloco é o próximo incremento.

Contexto. Tarefas de maior recompensa exigem **estruturas** de vários blocos. A `req` de uma task é uma lista de offsets relativos ao agente que vai submeter, p.ex. [`req(0, 1, b0)`, `req(0, 2, b1)`] — uma coluna de dois blocos ao sul do `submitter`. Completá-la muda qualitativamente o problema em relação ao bloco único.

Por que exige dois agentes (semântica do cenário). Pela especificação (`docs/scenario.md`):

- **attach só pega coisa adjacente** (distância 1, cardinal). Logo, um bloco exigido em $(0, 2)$ é **inalcançável** para um único agente — está a distância 2, e `rotate` preserva a distância ao agente, então não há como posicioná-lo sozinho.
- Os padrões 2-bloco gerados são **conexos** (o 2º bloco encosta no 1º, não no agente). Portanto a junção bloco-a-bloco só acontece via `connect`, que é **cooperativo entre dois agentes**.
- `connect(parceiro, x, y)`: ambos os agentes executam `connect` **no mesmo step**, cada um nomeando o outro e as **coords locais do seu próprio bloco**; os dois blocos precisam estar **adjacentes**. Após o sucesso, o bloco fica anexado aos **dois** agentes.
- `submit(task)`: exige o *submitter* **sobre uma goal zone** com **todos** os blocos do padrão anexados nos offsets exatos.

Protocolo proposto (montagem na goal zone). Decisão de projeto: usar uma **goal zone conhecida como ponto de encontro e montagem** — assim a estrutura já nasce onde o `submit` precisa ocorrer, eliminando um transporte final frágil no toro.

1. **Coordenador** seleciona uma task de 2 blocos cujos *dois* tipos tenham dispenser descoberto e cuja goal zone esteja mapeada; atribui papéis — **submitter** (p.ex. `explorador1`) e **helper** (`explorador2`) — o bloco de cada um (por índice da `req`) e a goal zone-alvo; anuncia tudo no `QuadroEquipe`.
2. **Cada worker** adota `worker`, busca seu bloco no dispenser do tipo (reusando a Fase A/B já validada) e o anexa.
3. **Rendezvous**: ambos navegam (por visão) até a goal zone designada e assumem a **configuração de acoplamento** exata. Exemplo para `[req(0, 1, b0), req(0, 2, b1)]`: o `submitter` S fica com `b0` em $(0, 1)$; o `helper` H se posiciona ao sul, com `b1` na célula que, vista de S, é $(0, 2)$ (i.e. os blocos (S_x, S_y+1) e (S_x, S_y+2) ficam adjacentes).
4. **connect sincronizado**: no mesmo step, S faz `connect(H, 0, 1)` e H faz `connect(S, 0, -1)` (cada um aponta o offset local do seu bloco). Sincronização via *handshake* no blackboard + uma **barreira** (ambos sinalizam "pronto para acoplar" e só então emitem `connect` no passo seguinte).
5. **detach + submit**: o helper faz `detach` do seu bloco (que permanece preso à estrutura de S via o `connect`); o submitter, já com o padrão completo sobre a goal zone, faz `submit(task)`.

Os três problemas difíceis e mitigações previstas.

- **Sincronização no mesmo step (connect/connect)**: o `connect` falha (`failed_partner`) se o parceiro não fizer `connect` no mesmo passo. Mitigação: estado compartilhado `pronto_para_connect(Agente, Task)` no `QuadroEquipe` + ambos só disparam `connect` no primeiro step em que **veem o parceiro pronto** (barreira de duas fases). Como o servidor executa as ações do step em ordem aleatória mas no mesmo step, basta a co-ocorrência.
- **Alinhamento preciso no grid toroidal**: posicionar dois agentes em células adjacentes específicas é frágil com coordenada absoluta (D1). Mitigação: alinhamento **por percepção relativa** — H enxerga S (são da mesma equipe e ficam próximos na goal zone) e ajusta seu offset por visão até a configuração de acoplamento, exatamente como o worker se posiciona no dispenser hoje.

- **Encontro (rendezvous):** os dois precisam chegar à *mesma* goal zone. Mitigação: a goal zone-alvo é **fixada pelo coordenador** (uma só, conhecida por ambos via blackboard); navegação por mapa+visão (camada já existente) leva ambos até lá.

O que precisaria mudar no código. (a) `QuadroEquipe`: anúncio rico da task multi-bloco (papéis, bloco por agente, goal zone-alvo) e o estado de barreira `pronto_para_connect`. (b) `coordenador.asl`: reconhecer `req` com 2 itens, checar dispensers dos dois tipos + goal zone, atribuir papéis. (c) `explorador.asl`: estender a máquina de estados do worker com o ramo multi-bloco (buscar bloco atribuído → ir à goal zone → alinhar por visão → barreira → connect → detach/submit). (d) **Bridge**: nada — connect, detach, rotate e disconnect **já estão expostos** como `@OPERATION` em `EISArtifact.java`.

Risco/avaliação. O acoplamento sincronizado de dois agentes com alinhamento no toro é, reconhecidamente, a parte de maior risco de fechar ao vivo dentro de um episódio (sincronização + posicionamento exato). O projeto acima decompõe o problema em etapas já validadas isoladamente (busca/anexo de bloco, navegação por visão, estado compartilhado no blackboard) mais o `connect` sincronizado como única peça genuinamente nova.

Como o exercício é atendido (mapeamento para o template do relatório)

Item do enunciado	Onde está no projeto
Item 1 — Introdução e objetivo	Time JaCaMo para o MAPC 2022 (<i>Agents Assemble II</i>). Objetivo: evoluir o esqueleto trivial (random walk + skip) para um time que descobre recursos, coordena-se e completa tarefas . Visão geral no topo deste README e em CLAUDE.md .
Item 2 — Análise e especificação do SMA	Organização em src/org/org.xml : 3 papéis sociais (<code>coordinator/explorer/worker</code>), esquema <code>completar_tarefa</code> , missões <code>m_coordinar/m_explorar/m_construir</code> , normas de obrigação. Especificação do problema físico (grid toroidal , percepção relativa , cadência por step) em D1 e em § Como o servidor é configurado .
Item 3 — Arquitetura e design do SMA	mapc2022.jcm (acoplamento agentes ↔ workspace ↔ organização), src/env/mapc/QuadroEquipe.java (blackboard CArtAgO compartilhado) e src/env/mapc/EISArtifact.java (ponte EISMASSim, uma <code>@OPERATION</code> por ação). Decisões de design registradas em D1–D4 .
Item 4 — Linguagens e plataforma	Jason (agentes BDI), CArtAgO (ambiente/artefatos), MOISE+ (organização), orquestrados por JaCaMo 1.3.0 ; ponte EISMASSim (TCP) ao servidor MASSim 2022.
Item 5 — Estratégia para o time	Implementada e validada ao vivo para tarefas de 1 bloco : exploração por cobertura serpentina (D3) ; memória de mapa relativa → absoluta + identificação de companheiros e fusão por <i>frame</i> de referência (passos 1–2); protocolo de tarefa <code>selecionar-anunciar-promover-adotar-buscar-submeter (D2)</code> , com preferência de tipo opcional no coordenador. Multi-bloco (<code>connect</code>) projetado em D4 . Lógica em

Item do enunciado	Onde está no projeto
	<code>src/agt/coordenador.asl</code> e <code>src/agt/explorador.asl</code> .
Item 6 — Características técnicas	Robustez validada: sincronização perceber ↔ agir (<code>await</code> no EISArtifact) e dedupe de actionID (resolveu ~74% de passos perdidos → ~0); anti-deadlock na coleta (<code>max_espera_bloco</code> + <code>timeout</code>); reset pós-submit ; navegação robusta ao toro (percepção relativa, D1); e tratamento do recorrente descasamento átomo×string na fronteira Jason/CArtAgO.
Item 7 — Discussão	Diferencial: uso explícito de MOISE+ , ausente em [5] LTI-USP e nos JaCaMo Builders 2020/21. Lições aprendidas: (a) o grid toroidal inviabiliza navegação por coordenada absoluta — a percepção relativa é a base correta; (b) o descasamento átomo×string entre o tipo da task (CArtAgO → string) e o percept (átomo) foi a causa-raiz recorrente que bloqueava a coleta; (c) só a validação ao vivo (não a inspeção do código) revelou esses defeitos.

Próximos passos sugeridos

Roadmap original do esqueleto (1–4). **Estado atual:** itens 1–4 implementados e validados ao vivo para tarefas de **1 bloco** (ver D2). O próximo incremento é o **multi-bloco** (item 5), cujo **projeto** está documentado em **D4** (semântica do connect, protocolo de montagem na goal zone, atribuição de papéis).

1. **Logs/memória de exploração** ✓ — `explorador.asl` registra dispensers e goal/role zones em coordenadas relativas à posição inicial, publicadas no QuadroEquipe. (paper [5] §3.1)
2. **Identificação de companheiros** ✓ — reconhecimento por posição-espelho + fusão de mapa por *frame* de referência. (paper [5] §3.2)
3. **Protocolo de tarefa** ✓ — coordenador seleciona/anuncia a task (modelo 2022: **sem** taskboard); exploradores se promovem a `worker`, adotam o papel de cenário, buscam e anexam o bloco.
4. **Submissão** ✓ — `request` → `attach` → `submit(NomeTask)` numa goal zone, validado para os três tipos de bloco (`b0/b1/b2`).
5. **Tarefas multi-bloco (connect)** — projetado em **D4**, ainda não implementado.

Resultados e conclusão

O que foi construído. Partindo de um esqueleto que apenas *conectava, percebia e agia* (com lógica trivial de *random walk* + `skip`), o time evoluiu para um SMA que **descobre recursos, constrói e funde um mapa compartilhado, identifica companheiros e executa o ciclo completo de uma tarefa de 1 bloco** no modelo MAPC 2022. A inteligência está nos agentes Jason (`coordenador.asl`, `explorador.asl`), o estado compartilhado no artefato CArtAgO (`QuadroEquipe.java`) e a organização em MOISE+ (`org.xml`).

O que foi validado ao vivo (cenário controlado `DemoConfig`, 30×30 — ver D2):

- Pipeline de tarefa **ponta a ponta**: `selecionar` → `anunciar` → `promover a worker` → `adotar papel` → `buscar bloco` → `request` → `attach` → `submit`, com `submit success`.
- **Generalidade aos três tipos de bloco** (`b0/b1/b2`), confirmando que a coleta não é *hardcoded* (em um episódio, **os dois** workers completaram a tarefa de `b2`).
- **Eliminação de ~74% de passos perdidos** (`no_action`) via sincronização perceber ↔ agir, e **navegação estável no grid toroidal** (sem o *runaway* de posição que corrompia o mapa).

Contribuições técnicas e lições. (a) Em um grid **toroidal de tamanho desconhecido**, a navegação correta é por **percepção relativa** a alvos visíveis, com o mapa absoluto servindo só como camada subsidiária (**D1**). (b) A fronteira **Jason** ↔ **CARTAgO** reintroduz silenciosamente o descasamento **átomo×string** (tipo da task como `"b1"` vs. `percept b1`) — foi a causa-raiz que bloqueava a coleta, resolvida convertendo o tipo a átomo na origem. (c) Defeitos como esses **só apareceram em execução ao vivo**, não na inspeção do código — daí o investimento em instrumentação e num cenário de teste determinístico.

Limitações e trabalho futuro. O pipeline cobre tarefas de **1 bloco**; tarefas **multi-bloco** (que exigem `connect` cooperativo e sincronizado entre dois agentes) estão **projetadas mas não implementadas (D4)**. Outras frentes: leilão entre workers pela proximidade ao dispenser, uso de `clear/digger` para desencurrular, e estimativa do tamanho do grid para coordenadas modulares.

Referências consultadas

- Stabile Jr., M.F.; Sichman, J.S. *The LTI-USP Strategy to the 2020/2021 MAPC*. LNAI 12947, 2021. — referência [5] do enunciado.
- Amaral, C.J. et al. *JaCaMo Builders: Team Description for the MAPC 2020/21*. LNAI 12947, 2021. — discutido por usar JaCaMo+CARTAgO mas **sem MOISE+**, gap que este projeto preenche.
- Documentação oficial do cenário: [../massim_2022/docs/scenario.md](https://github.com/massim2022/docs/blob/main/scenario.md)